ECE 350: Real-Time Operating Systems Winter 2027

Lecture 21 — Power Management and Energy-Aware Scheduling

Jeff Zarnett *2026-09-06*

We Got The Power

Motivating why we should care about power consumption is easy: the consumer market for buying laptops that last only an hour is tiny, if it exists at all. Similarly, you would not be very happy with a phone that doesn't make it through the average day. In fact, if your phone is not lasting long enough, you probably have decided you need either a new phone or, at least, a new battery. Even if we're not talking about running on battery, electricity has some cost to it. Energy is a resource, and something the operating system can manage (if it's aware of it). Operating systems can discover and configure power states through a standard interface called *ACPI*: Advanced Configuration and Power Interface. This has existed for decades now.

While it might be possible to manage power consumption across different devices in the system, given the limited time available to us in this course, we'll restrict the discussion to managing the power state of the CPU.

ACPI States. It's also important to differentiate this discussion from the general ACPI states of the device (i.e., the laptop as a whole). The ACPI states are the ones the user typically thinks of: powered off, powered on, sleeping, and hibernating (off but state stored to disk to restore on wake-up).

If you close the lid of your laptop without selecting to shut down or power-off, it probably goes to a "sleep" mode. If it thinks that battery is getting low, it might transition to a "hibernate" mode by writing the contents of RAM to disk to restore when power comes back.

In this topic, we are considering the ACPI state to be that the computer is powered on.

Lay in a Course, Maximum Warp

There are certainly times where we want to use the CPU to its maximum power. If the laptop is plugged in to the charger and the user has a very CPU-intensive task running (play a game, render a video, execute a simulation, etc.) then it makes sense to go as hard as we can. Running the CPU at its maximum power means it delivers the best performance. But even in Star Trek, they don't go everywhere at Maximum Warp, do they?

You already know one reason why we should not run the CPU at 100% at all times from the title of this topic: energy consumption and therefore battery life. The other reason, of course, is heat. Heat can be uncomfortable if the laptop is in your lap or the phone in your hand, but heat is also the enemy of reliability. Running the components very hot will shorten their lifespan and leads to increased errors in calculations – but we'll come to that in the reliability topic.

Maybe a helpful way of looking at it: for the CPU to be at 100% is like running everywhere you go. Run to class, run to the lab, run to lunch... You *can* do it, sure, but this depletes your energy way faster than walking and on a hot day you will definitely start feeling the negative effects of the heat. There's a time and place for it, but you don't run all the time. Instead, you adapt your pace of travel to the urgency of the situation and the state of resources. Want to go to class? Leisurely pace. Going to miss your bus? Jog for it, but not at max effort. Late for the exam? Maximum warp, Mr. Sulu.

The CPU has what are referred to as P-States and C-States. P is for "performance" and C is for "CPU idle". Broadly, P-States control the voltage and CPU frequency and C-States determine what the CPU does to reduce power consumption when no code is running (idle). For this section on P- and C-states, we'll dive into this helpful

guide [Bal18] that builds on some Intel Xeon data sheets, developer guides, and some kernel sources. Other CPUs with the same or other architectures also have comparable power management features, so the concepts here apply even if there are different names of things and different levels.

So, we have two levers available to reduce power consumption. Performance level and turning things off.

P-States. The first is the performance level setting, which says what clock speed we want to run the CPU at. This works because, to a first approximation, the power consumption of the CPU is linear with respect to frequency and quadratic with respect to voltage (the source writes it as $P \propto fV^2$ which is oversimplifying but directionally correct). A small amount of hardware knowledge helps in explaining why.

Your first-year electromagnetism course includes the formula $P = V \times I$ so increasing the voltage very obviously would increase the power consumption. The less-obvious component of this is that increasing the frequency is going to require a higher voltage to drive the components faster. This is, not coincidentally, why CPU clock speeds today are about the same as they were in 2005 (yes, Intel had a 3.8 GHz processor then!): going faster requires more voltage and more power and it quickly becomes a huge problem in terms of heat, power draw, error rates, and battery life. So CPU manufacturers have stopped chasing higher clock speeds.

P states are numbered: `P0` is maximum frequency; any of the other numbers represent one of the lower frequency options offered by this hardware (the spec says a higher number is slower speed). The more modern multi-core CPUs permit having different power states for different cores, and thus there exists the idea of power states for the package (all cores) and for individual cores. But let's not get bogged down in that for now.

So, one way that we can immediately see improvement on power consumption is to set a higher-numbered P-state for the CPU and therefore run the CPU slower. Some devices do this automatically, e.g., a laptop moves to a lower-power state when unplugged from the charger. Or it can be user-initiated, when they choose the "low power" or "battery saver" mode. Some devices – including a previous phone I had – gave you the option of more than one battery-saver level which you can imagine corresponds to different P-state settings.

Now, on your phone (and probably other devices) it's more complicated than this, because going to low power mode changes more than the clock speed of the CPU. It also changes things like how often apps are permitted to refresh data in the background. We'll come back to that later in this topic.

Some CPUs also offer "boost" power where clock speed exceeds the nominal maximum for short periods of time. This only works when the chip is cool enough to do this without exceeding its heat limit, because the heat increases quite rapidly in this mode. This can, however, provide a meaningful performance boost that reduces latency if the workload is short but intense.

C-States. C-States are a lot simpler conceptually, because they are simply shutting down parts of the processor that are not currently being used. Akin to turning off the lights when you are not in a given room, or turning off wifi when you're out in the woods far away from any wifi access points, so that your phone doesn't waste any energy searching for a connection it will never find.

C-States are numbered, with `C0` meaning everything is turned on and actively executing instructions. Any higher number than that means turning some things off. The ACPI standard 6.4 [1] describes states C1, C2, and C3:

- C0: CPU is executing instructions and running normally.
- C1: CPU is halted (not executing instructions) but maintains cache and can resume very quickly if needed.
- C2: Lower-power than C1 and takes more time and higher transition energy cost to wake back up to C0 than from C1.

[1] `https://uefi.org/htmlspecs/ACPI_Spec_6_4_html/08_Processor_Configuration_and_Control/processor-power-states.html`

- C3: Even lower-power than C2, caches not kept up to date, slowest time and highest energy cost to return to C0 state.

As with the P-states, modern multicore CPUs often have many more states than the minimum the spec requires, and differentiate between package-level and core-level C states. It's hard to do a real demo of this because it requires loading a kernel module, but we might be able to get an idea of it by looking at this CoreFreq GitHub project: `https://github.com/cyring/CoreFreq`.

Can C-states be disabled to reduce latency? Yes, many CPUs do permit this option – or at the very least, we can configure the operating system not to issue the commands that order a shift to a lower-power state.

Discussion question: should we go fast temporarily and get to the idle state sooner, or go slower for a longer period of time? What factors would we need to know to answer that? You can imagine this is perhaps viewed as $t_1 \times e_1$ ("race to idle") vs $t_2 \times e_2$ ("slowdown") where t_1 is the time to run at full power and e_1 is the energy cost for running at full power; and t_2 is the time to run at a lower power and e_2 that energy cost. It's tempting to say that lower and slower produces the best results (this might be true for cooking), but that's not always true.

Based on a research paper [DMAH16], it is of course the case that there's variation based on workload. Customizing the approach, the authors claim, can net something like 13-18% energy savings.

One of the most important determining factors is leakage: how much power is burned for no good purpose; that is, due to electricity flowing through places where, in the ideal model of the transistor, it would not flow at all. Intel CPUs, allegedly, are big offenders in this area. For memory-bound work, slowdown might be better, though.

If there is demand for the CPU, then the operating system may need to wake up a sleeping core (send it back to the C0 state). It can similarly send things to sleep with instructions. Hardware interrupts can also wake up a sleeping core. So when do we send cores to sleep? That's a really interesting topic.

Energy-Aware Scheduling

Energy-Aware Scheduling is, unsurprisingly, exactly what it sounds like: it's the scheduler making choices that reduce energy usage. We can break this down into a few different areas: sending cores to sleep & waking them up, choosing which cores are active at all, and managing background data & refreshes.

It's Bedtime. The OS can issue commands to the cores to send them to sleep; the idle task, for example, may contain the `HLT` (halt) instruction which puts it in C1. The OS can also use other instructions to send cores to other states above C1. The cost of waking up from C1 is negligible, but it's more expensive to come back from other states. So we would not send cores to this state unless we are reasonably sure that we won't be bringing it back for at least a little bit.

Putting a core to sleep (C1 or above) means the scheduler should not schedule threads to run on that core because it's not really ready to run threads right now. If demand for CPU is increasing and this seems like more than just a transient blip, it makes sense to re-activate some cores – starting first with the ones in C1 state and bringing up the other cores only if truly needed.

The specification makes clear that interrupts (the hardware kind) are a trigger to send a core back to the C0 state if it needs to be handled. More advanced CPUs can route hardware interrupts to core(s) that are already in the C0 state to reduce the impact on energy.

This covers the what, but there remains the question of how. The kernel docs[2]reference the idea of *governors*, saying "The role of the governor is to find an idle state most suitable for the conditions at hand.". The description of each idle state, from the view of the governor, involves two figures: target residency and worst-case exit latency.

[2] `https://docs.kernel.org/admin-guide/pm/cpuidle.html`

Those are, respectively, the minimum time the hardware needs to spend in that state, which includes the time to enter that low power state, and the maximum time it could take to bounce back to C0 from that state.

Given this information, the governor can likely rule out some states immediately from consideration if it thinks that the idle period is likely to be short. If the expected idle period is less than the sum of these two times, then, it's impractical to enter into that state; choose another one with smaller numbers. Predicting when there will be more work to do is difficult, of course. The kernel knows about internal timers – e.g., time slicing – so it can use that as input, but other, unexpected things can wake up the CPU.

Actually, the scheduler interrupt is something that the kernel should handle differently than other timers. A different timer is for a specific event to be finished, but the scheduler one is specifically to run the scheduler. If there's nothing to do and CPUs are idle, then waking up the low-power CPU just for it to run the scheduler and decide to go back to sleep is a little wasteful. You may or may not enjoy hitting snooze on your alarm six times before getting out of bed, but I'm guessing that is not optimal for sleep. So the kernel doesn't do the scheduler interrupt on those idle CPUs (usually).

There are multiple different governor implementations supported, but we'll restrict ourselves to just one example, called the `menu` governor. It uses past data to predict the idle duration, and then uses the output of that calculation to select the best idle state. The kernel docs explain how the prediction is calculated: use the last 8 observed idle duration values; if the variance is small or small relative to the average, use that value, otherwise drop outliers and try again. There are more components to the algorithm than this, including a correction factor, but let's not get bogged down in the details; the part that should not be hand-waved away is how to get an estimate the length of a sleep and the answer is recent history. Once that estimate is reached, then the governor can look at the list of idle states and choose the one that is the best match based on the target residency and exit latency. The approach of using past behaviour to come up with an estimate of the future idle state is something we've observed in a few situations and is a common pattern we've used in other scheduling algorithms before.

Big Cores and Little Cores. The kind of CPU organization in this course is and remains Symmetric Multiprocessing (SMP), but there's a little twist on this that is common in mobile devices: heterogeneous CPU topology, which is fancy words for saying we have cores of different size and power. It's still SMP in the absolute sense because they are general-purpose CPUs and have their own private ready queues and all that. They are just not identical in all ways. This is not a niche thing: modern phones like the Samsung Galaxy S26 follow this strategy and have more than two. In this topic, we will just consider big (high performance, high power) and little (lower performance, lower power), even if real devices might have more than two options.

The Linux kernel documentation about this topic[3] provides a good guide for how to handle this situation and we'll use that. It's a little dense to read but a high-level description of the logic follows.

The scheduler effectively has to guess where it should assign work to minimize the energy consumption without lowering performance (too much). It is easy enough for the kernel to have a good idea of what the capacity (performance capabilities) is for each kind of core. Similarly, the kernel has plenty of information about how much work there is to do right now and how busy each core is. The unknown variable is, how big is the thing to do?

If the task is estimated to be "too big", it will have to go to a big core and not the small ones. Sensible. But for smaller tasks, they could go to either one. Much like the race-to-idle question, we have to ask the question about whether it makes more sense to run the task on a smaller core for a longer time or on the larger core for a shorter time. Although I did not find a specific research paper to back this up (okay, I also did not look for one), it is almost certainly task-behaviour dependent: CPU-bound or I/O-bound.

The kernel implementers seem to agree with that assessment, because, unsurprisingly, the strategy to guess what the energy consumption would be is to look at the utilization average, i.e., when this thread has previously been selected to run, how much CPU time has it consumed on average? With a normalization factor to account for the fact that time on a small CPU is not exactly the same as time on a big CPU. The best predictor of future behaviour

[3] `https://www.kernel.org/doc/html/v5.8/scheduler/sched-energy.html`

is, once again, past behaviour.

Let's imagine the specific task being considered now is small enough that it could go in a little or big core. The scheduler will find the CPU with the lowest usage in each of the size classes. For each of those, guess what the energy consumption would be, and select the one with the lowest predicted cost.

There's a threshold where the kernel stops trying to do the energy-aware scheduling: 80% utilization. Once demand is that high, trying to save energy is out the window since it probably won't help much and wastes time.

Hold On Now. User control is sometimes possible! In the settings of your phone you can choose whether apps are permitted to refresh in the background or if they can use cellular data (or only wifi). You could do this intentionally to save battery – but you probably don't want to limit it for everything. Turning it off for social media seems sensible – when you open the app it will show you whatever ads it was going to show you anyway – but you don't want to miss important notifications from messaging apps.

There is also sometimes a middle setting that defers work until later; iOS, for example, introduced the ability for background tasks to be deferred until later, such as when the device is plugged into the charger [4]. The nature of the task determines what conditions need to be true to execute the deferred task: presence of WiFi, being on the charger, time of day, et cetera.

The Android developer guide does suggest batching or bundling operations. If the radio of your mobile device is off, it takes some time and energy to turn it on and connect. So a given app would be more energy-efficient if it groups up requests, so that the cost of powering up the radio is minimized.

That's voluntary, though. The docs for the Android "Doze" feature describe how it will restrict data (forcibly) to designated windows. In practice, this also gives the benefit of reducing the number of times the radio powers up and therefore reducing energy consumption. I did not find anything explicitly declaring an equivalent feature in iOS, but Apple is occasionally pretty quiet about the internals of things, so it may exist.

The ability to restrict background data is not only for power management; it can also control the consumption of data when the connection is metered. If there is a dollar price per gigabyte (or even a speed slowdown beyond some cliff), it might be the user's preference to restrict background data to stop an app from chewing up their monthly usage unnecessarily.

How Much Precision? Timers are usually not super precise in mobile systems either; some amount of fuzziness permits lining things up. This means that if a wake-up was meant to happen at time x and then the system would go back to idle and then immediately wake up at $x+\epsilon$, the OS will nudge the first timer back a bit so both wake-ups happen together.

This is not insane, given what we know of the semantics of `sleep()` or `usleep()` as a system call: it puts the thread to sleep for *at least* the duration provided as the argument but it could be longer. And on top of that, when the process or thread is woken up by the expiration of the timer, there are no guarantees about when that thread is selected to run by the scheduler anyway. So even if the thread is woken up exactly at time x it could easily have been $x + \epsilon$ before it runs. The size of the nudge the OS is willing to make is probably larger on a mobile device than on a desktop one, but that's just a question of degree, not legality.

If you are thinking that sounds imprecise and you need timers to be more exact than that – that level of precision is for real-time systems! Which the user-accessible OS of the mobile device is not (even if the radio might contain an embedded one).

Background Killing. Mobile operating systems also kill background processes to stop them from running. This guarantees that the app isn't executing in the background and isn't using data. Since apps are rarely closed on the phone (yes, you can, but you have to go out of your way to do this), the OS is aggressive about this. You have

[4] We can see this in a 2019 WWDC talk `https://developer.apple.com/videos/play/wwdc2019/707/?time=942`

probably experienced a symptom of this if you've been away from your mobile browser for a bit and when you switch to it, the page re-loads.

Both iOS and Android have CPU-usage-based background task killing. You could argue that this is also a nice security feature because it means that advertised mobile loot-box (gambling) "game" (if I could make those quotes bigger, I would) can't also mine cryptocurrency on your phone while it's in the background. However, sometimes your app can ask to opt out of these things. That may have legitimate uses – training a machine learning model, for example – but it would be reasonable to be suspicious of an app that asked permission to do that.

Still, this antagonizes some app developers! There is a whole website dedicated to complaining about Android devices aggressively terminating apps. This is, as far as I can see, another one of those "some people are jerks" problems: if all app developers behaved nicely, then there would be no need for automatic killing or CPU-usage restrictions. But alas, left to their own devices people will do things that hurt overall performance. Rightly or wrongly, it is sometimes the device vendor that's blamed for poor battery life in these circumstances.

References

[Bal18] Mete Balci. A minimum complete tutorial of cpu power management, c-states and p-states, 2018. Online; accessed 4-July-2026. URL: `https://metebalci.com/blog/a-minimum-complete-tutorial-of-cpu-power-management-c-states-and-p-states/`.

[DMAH16] Anup Das, Geoff V. Merrett, and Bashir M. Al-Hashimi. The slowdown or race-to-idle question: Workload-aware energy optimization of smt multicore platforms under process variation. pages 535–538, 2016.